

```
'use strict';  
/**  
 * VLF – serveur MVP (version fichier  
unique, pour déploiement simplifié)  
 * 100% Node.js natif : aucune dépendance à  
installer.  
 * Démarrage : node server.js  
 *  
 * Ce fichier regroupe volontairement tout  
le backend (base de données,  
authentification,  
 * routes API, serveur HTTP) en un seul  
fichier pour faciliter un déploiement  
manuel  
 * (ex. copier-coller dans l'éditeur web de  
GitHub) sans avoir à gérer plusieurs  
dossiers.  
 * La version "modulaire" (plusieurs  
fichiers) reste disponible séparément pour  
le  
 * développement continu.  
 */  
const http = require('node:http');  
const fs = require('node:fs');  
const path = require('node:path');  
const { URL } = require('node:url');  
const crypto = require('node:crypto');  
const { DatabaseSync } =  
require('node:sqlite');
```

```

const PORT = process.env.PORT ?
Number(process.env.PORT) : 3000;
const PUBLIC_DIR = path.join(__dirname,
'public');
const IS_PROD = process.env.NODE_ENV ===
'production';
const DB_PATH = process.env.VLF_DB_PATH ||
path.join(__dirname, 'vlf.sqlite');

//
=====
// BASE DE DONNÉES (schéma + seed) – voir
cahier des charges, section 12
//
=====
const db = new DatabaseSync(DB_PATH);
db.exec('PRAGMA foreign_keys = ON;');

db.exec(`
CREATE TABLE IF NOT EXISTS users (
  id TEXT PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  password_salt TEXT NOT NULL,
  nom TEXT,
  pays_residence TEXT,
  nationalite TEXT,
  is_admin INTEGER NOT NULL DEFAULT 0,
  created_at TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS sessions (

```

```
token TEXT PRIMARY KEY,  
user_id TEXT NOT NULL REFERENCES  
users(id) ON DELETE CASCADE,  
created_at TEXT NOT NULL,  
expires_at TEXT NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS profiles (  
user_id TEXT PRIMARY KEY REFERENCES  
users(id) ON DELETE CASCADE,  
situation_familiale TEXT,  
situation_emploi TEXT,  
situation_admin TEXT,  
updated_at TEXT NOT NULL  
);  
  
CREATE TABLE IF NOT EXISTS sources (  
id TEXT PRIMARY KEY,  
organisme TEXT NOT NULL,  
reference TEXT,  
url TEXT,  
date_verification TEXT NOT NULL,  
statut TEXT NOT NULL DEFAULT 'a_jour'  
);  
  
CREATE TABLE IF NOT EXISTS procedures (  
id TEXT PRIMARY KEY,  
nom TEXT NOT NULL,  
categorie TEXT NOT NULL,  
conditions_json TEXT,  
source_id TEXT REFERENCES sources(id),  
date_verification TEXT NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS documents (  
  id TEXT PRIMARY KEY,  
  type TEXT NOT NULL,  
  procedure_id TEXT REFERENCES  
procedures(id) ON DELETE CASCADE,  
  obligatoire INTEGER NOT NULL DEFAULT 1  
);
```

```
CREATE TABLE IF NOT EXISTS results (  
  id TEXT PRIMARY KEY,  
  user_id TEXT NOT NULL REFERENCES  
users(id) ON DELETE CASCADE,  
  objectif TEXT,  
  situation_json TEXT NOT NULL,  
  main_path TEXT,  
  autonomy_level TEXT,  
  documents_json TEXT,  
  created_at TEXT NOT NULL,  
  updated_at TEXT NOT NULL  
);
```

```
CREATE INDEX IF NOT EXISTS  
idx_sessions_user ON sessions(user_id);  
CREATE INDEX IF NOT EXISTS idx_results_user  
ON results(user_id);  
CREATE INDEX IF NOT EXISTS  
idx_procedures_categorie ON  
procedures(categorie);  
`);
```

```
function seedIfEmpty() {  
  const countRow = db.prepare('SELECT  
COUNT(*) AS n FROM sources').get();  
  if (countRow.n > 0) return;
```

```

    const now = new
Date().toISOString().slice(0, 10);
    const insertSource = db.prepare(`INSERT
INTO sources (id, organisme, reference,
url, date_verification, statut) VALUES
(?,?,?, ?, ?, ?)`);
    const insertProcedure =
db.prepare(`INSERT INTO procedures (id,
nom, categorie, conditions_json, source_id,
date_verification) VALUES (?,?,?,?,?,?)`);
    const insertDocument = db.prepare(`INSERT
INTO documents (id, type, procedure_id,
obligatoire) VALUES (?,?,?,?)`);

    const sources = [
        ['src_service_public', 'Service-
Public.fr', "Portail officiel de
l'administration française",
'https://www.service-public.fr', now,
'a_jour'],
        ['src_anef', 'ANEF', 'Administration
numérique des étrangers en France',
'https://administration-etrangers-en-
france.interieur.gouv.fr', now, 'a_jour'],
        ['src_france_visas', 'France-Visas',
'Portail officiel des visas',
'https://france-visas.gouv.fr', now,
'a_jour'],
        ['src_legifrance', 'Légifrance',
'CESEDA et textes en vigueur',
'https://www.legifrance.gouv.fr', now,
'a_verifier'],
        ['src_france_travail', 'France

```

```

Travail', 'Répertoire ROME et listes de
métiers en tension',
'https://www.francetravail.fr', now,
'a_jour'],
];
for (const s of sources)
insertSource.run(...s);

const procedures = [
  ['proc_travail_salarie', 'Recrutement
professionnel (salarié)', 'travail',
  JSON.stringify(['Contrat de travail
ou promesse d\'embauche', 'Vérification de
la correspondance ROME', 'Vérification de
la zone en tension'])],
  'src_france_travail', now],
  ['proc_passeport_talent', 'Passeport
talent', 'travail',
  JSON.stringify(['Profil qualifié ou
projet économique', 'Diplôme ou expérience
significative'])],
  'src_france_visas', now],
  ['proc_famille_francais', 'Famille de
Français (conjoint, enfant, ascendant)',
  'famille',
  JSON.stringify(['Lien familial avec
un ressortissant français établi',
  'Justificatif de nationalité française de
la personne rejointe'])],
  'src_service_public', now],
  ['proc_regroupement_familial',
  'Regroupement familial classique',
  'famille',
  JSON.stringify(['Ressources

```

```
suffisantes', 'Logement adapté', 'Personne  
rejointe en séjour régulier hors UE']],  
    'src_service_public', now],  
    ['proc_membre_famille_ue', "Membre de  
famille d'un citoyen de l'Union",  
    'famille',  
    JSON.stringify(['Lien familial avec  
un citoyen UE/EEE/Suisse', 'Nationalité  
UE/EEE/Suisse de la personne accompagnée  
vérifiée']),  
    'src_service_public', now],  
    ['proc_nationalite_mariage',  
    'Nationalité par mariage', 'nationalite',  
    JSON.stringify(['Durée de mariage  
requis', 'Communauté de vie effective']),  
    'src_legifrance', now],  
    ['proc_nationalite_naturalisation',  
    'Naturalisation', 'nationalite',  
    JSON.stringify(['Durée de résidence  
en France', "Condition d'assimilation",  
    'Ressources stables']),  
    'src_legifrance', now],  
    ['proc_anef_bloque', 'Suivi de blocage  
ANEF', 'anef',  
    JSON.stringify(['Date de dépôt',  
    'Statut ANEF actuel', 'Silence > 4 mois =  
rejet implicite dans la plupart des cas']),  
    'src_anef', now],  
    ['proc_visa_visiteur', 'Visa long  
séjour visiteur', 'visa',  
    JSON.stringify(['Ressources  
suffisantes', 'Assurance maladie',  
    'Engagement à ne pas travailler']),  
    'src_france_visas', now],
```

```

    ['proc_visa_medical', 'Visa pour soins
médicaux', 'visa',
    JSON.stringify(['Certificat médical
détaillé', "Prise en charge hospitalière ou
attestation d'admission", 'Garantie
financière'])],
    'src_france_visas', now],
];
for (const p of procedures)
insertProcedure.run(...p);

const documents = [
    ['doc1', 'Passeport en cours de
validité', 'proc_travail_salarie', 1],
    ['doc2', 'CV', 'proc_travail_salarie',
1],
    ['doc3', "Contrat de travail ou
promesse d'embauche",
'proc_travail_salarie', 0],
    ['doc4', 'Passeport en cours de
validité', 'proc_famille_francais', 1],
    ['doc5', 'Acte de mariage ou de
naissance établissant le lien',
'proc_famille_francais', 1],
    ['doc6', 'Justificatif de nationalité
française de la personne rejointe',
'proc_famille_francais', 1],
    ['doc7', 'Récépissé ou accusé de
dépôt', 'proc_anef_bloque', 1],
    ['doc8', "Historique des échanges avec
l'administration", 'proc_anef_bloque', 0],
    ['doc9', 'Certificat médical détaillé',
'proc_visa_medical', 1],
    ['doc10', 'Justificatif de prise en

```



```

charge financière', 'proc_visa_medical',
1],
];
for (const d of documents)
insertDocument.run(...d);

const demoSalt =
crypto.randomBytes(16).toString('hex');
const demoHash =
crypto.scryptSync('demo1234', demoSalt,
64).toString('hex');
db.prepare(`INSERT INTO users (id, email,
password_hash, password_salt, nom,
pays_residence, nationalite, is_admin,
created_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`)
.run('user_demo', 'demo@vlf.fr',
demoHash, demoSalt, 'Utilisateur Démo',
'FR', 'FR', 0, new Date().toISOString());

const adminSalt =
crypto.randomBytes(16).toString('hex');
const adminHash =
crypto.scryptSync('admin1234', adminSalt,
64).toString('hex');
db.prepare(`INSERT INTO users (id, email,
password_hash, password_salt, nom,
pays_residence, nationalite, is_admin,
created_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`)
.run('user_admin', 'admin@vlf.fr',
adminHash, adminSalt, 'Administrateur VLF',
'FR', 'FR', 1, new Date().toISOString());
}
seedIfEmpty();

```

```
//
=====
=====
// AUTHENTICATION
//
=====
=====
const SESSION_DURATION_MS = 1000 * 60 * 60
* 24 * 14; // 14 jours

function hashPassword(password) {
  const salt =
crypto.randomBytes(16).toString('hex');
  const hash = crypto.scryptSync(password,
salt, 64).toString('hex');
  return { hash, salt };
}
function verifyPassword(password, hash,
salt) {
  const candidate =
crypto.scryptSync(password, salt,
64).toString('hex');
  const a = Buffer.from(candidate, 'hex');
  const b = Buffer.from(hash, 'hex');
  if (a.length !== b.length) return false;
  return crypto.timingSafeEqual(a, b);
}
function createSession(userId) {
  const token =
crypto.randomBytes(32).toString('hex');
  const now = new Date();
  const expires = new Date(now.getTime() +
SESSION_DURATION_MS);
  db.prepare(`INSERT INTO sessions (token,
```

```

user_id, created_at, expires_at) VALUES
(?,?,?,?)`)
    .run(token, userId, now.toISOString(),
expires.toISOString());
    return { token, expires };
}
function getUserBySessionToken(token) {
    if (!token) return null;
    const row = db.prepare(`SELECT
s.expires_at, u.* FROM sessions s JOIN
users u ON u.id = s.user_id WHERE s.token =
?`).get(token);
    if (!row) return null;
    if (new Date(row.expires_at).getTime() <
Date.now()) {
        db.prepare(`DELETE FROM sessions WHERE
token = ?`).run(token);
        return null;
    }
    const { password_hash, password_salt,
expires_at, ...safeUser } = row;
    return safeUser;
}
function destroySession(token) {
    if (!token) return;
    db.prepare(`DELETE FROM sessions WHERE
token = ?`).run(token);
}
function parseCookies(req) {
    const header = req.headers.cookie;
    const out = {};
    if (!header) return out;
    header.split(';').forEach((pair) => {
        const idx = pair.indexOf('=');

```

```

        if (idx === -1) return;
        out[pair.slice(0, idx).trim()] =
decodeURIComponent(pair.slice(idx +
1).trim());
    });
    return out;
}
function setSessionCookie(res, token,
expires) {
    const attrs =
['vlf_session=${encodeURIComponent(token)}'
, 'Path=/', 'HttpOnly', 'SameSite=Lax',
`Expires=${expires.toUTCString()}`];
    if (IS_PROD) attrs.push('Secure');
    res.setHeader('Set-Cookie', attrs.join(';
'));
}
function clearSessionCookie(res) {
    const attrs = ['vlf_session=', 'Path=/',
'HttpOnly', 'SameSite=Lax', 'Expires=Thu,
01 Jan 1970 00:00:00 GMT'];
    if (IS_PROD) attrs.push('Secure');
    res.setHeader('Set-Cookie', attrs.join(';
'));
}

//
=====
=====
// HELPERS API
//
=====
=====
function sendJson(res, statusCode, payload)

```

```

{
  const body = JSON.stringify(payload);
  res.writeHead(statusCode, { 'Content-
Type': 'application/json; charset=utf-8',
'Content-Length': Buffer.byteLength(body)
});
  res.end(body);
}
function readJsonBody(req) {
  return new Promise((resolve, reject) => {
    let data = '';
    let size = 0;
    const MAX = 1_000_000;
    req.on('data', (chunk) => {
      size += chunk.length;
      if (size > MAX) { reject(new
Error('Payload trop volumineux'));
req.destroy(); return; }
      data += chunk;
    });
    req.on('end', () => {
      if (!data) return resolve({});
      try { resolve(JSON.parse(data)); }
catch (e) { reject(new Error('JSON
invalide')); }
    });
    req.on('error', reject);
  });
}
function isValidEmail(email) { return
typeof email === 'string' &&
/^[^\\s@]+@[^\\s@]+\\.^[^\\s@]+$/\\.test(email); }
function requireAuth(ctx, res) {
  if (!ctx.user) { sendJson(res, 401, {

```

```

error: 'Authentication requise.' });
return false; }
  return true;
}
function requireAdmin(ctx, res) {
  if (!requireAuth(ctx, res)) return false;
  if (!ctx.user.is_admin) { sendJson(res,
403, { error: 'Accès réservé aux
administrateurs.' }); return false; }
  return true;
}

//
=====
=====
// HANDLERS – auth
//
=====
=====
async function handleSignup(req, res) {
  let body;
  try { body = await readJsonBody(req); }
catch (e) { return sendJson(res, 400, {
error: e.message }); }
  const { email, password, nom,
pays_residence, nationalite } = body;
  if (!isValidEmail(email)) return
sendJson(res, 400, { error: 'Adresse email
invalide.' });
  if (typeof password !== 'string' ||
password.length < 8) return sendJson(res,
400, { error: 'Le mot de passe doit
contenir au moins 8 caractères.' });
  const existing = db.prepare('SELECT id

```

```

FROM users WHERE email =
?').get(email.toLowerCase());
    if (existing) return sendJson(res, 409, {
error: 'Un compte existe déjà avec cet
email.' });
    const { hash, salt } =
hashPassword(password);
    const id = 'user_' +
crypto.randomBytes(10).toString('hex');
    const now = new Date().toISOString();
    db.prepare(`INSERT INTO users (id, email,
password_hash, password_salt, nom,
pays_residence, nationalite, is_admin,
created_at) VALUES (?, ?, ?, ?, ?, ?, ?, 0, ?)`)
    .run(id, email.toLowerCase(), hash,
salt, nom || null, pays_residence || null,
nationalite || null, now);
    const { token, expires } =
createSession(id);
    setSessionCookie(res, token, expires);
    return sendJson(res, 201, { id, email:
email.toLowerCase(), nom: nom || null });
}
async function handleLogin(req, res) {
    let body;
    try { body = await readJsonBody(req); }
    catch (e) { return sendJson(res, 400, {
error: e.message }); }
    const { email, password } = body;
    if (!isValidEmail(email) || typeof
password !== 'string') return sendJson(res,
400, { error: 'Identifiants invalides.' });
    const user = db.prepare('SELECT * FROM
users WHERE email =

```

```

?').get(email.toLowerCase());
  if (!user || !verifyPassword(password,
user.password_hash, user.password_salt))
return sendJson(res, 401, { error: 'Email
ou mot de passe incorrect.' });
  const { token, expires } =
createSession(user.id);
  setSessionCookie(res, token, expires);
  return sendJson(res, 200, { id: user.id,
email: user.email, nom: user.nom });
}
function handleLogout(req, res, ctx) {
  const cookies = parseCookies(req);
  destroySession(cookies.vlf_session);
  clearSessionCookie(res);
  return sendJson(res, 200, { ok: true });
}
function handleMe(req, res, ctx) {
  return sendJson(res, 200, { user:
ctx.user || null });
}
async function handleChangePassword(req,
res, ctx) {
  if (!requireAuth(ctx, res)) return;
  let body;
  try { body = await readJsonBody(req); }
catch (e) { return sendJson(res, 400, {
error: e.message }); }
  const { currentPassword, newPassword } =
body;
  if (typeof newPassword !== 'string' ||
newPassword.length < 8) return
sendJson(res, 400, { error: 'Le nouveau mot
de passe doit contenir au moins 8

```



```

caractères.' });
    const row = db.prepare('SELECT * FROM
users WHERE id = ?').get(ctx.user.id);
    if (!row ||
!verifyPassword(currentPassword || '',
row.password_hash, row.password_salt))
return sendJson(res, 401, { error: 'Mot de
passe actuel incorrect.' });
    const { hash, salt } =
hashPassword(newPassword);
    db.prepare('UPDATE users SET
password_hash = ?, password_salt = ? WHERE
id = ?').run(hash, salt, ctx.user.id);
    const cookies = parseCookies(req);
    db.prepare('DELETE FROM sessions WHERE
user_id = ? AND token !=
?').run(ctx.user.id, cookies.vlf_session ||
'');
    return sendJson(res, 200, { ok: true });
}

//
=====
=====
// HANDLERS – profil, procédures, sources,
résultats
//
=====
=====
async function handleUpdateProfile(req,
res, ctx) {
    if (!requireAuth(ctx, res)) return;
    let body;
    try { body = await readJsonBody(req); }

```

```

catch (e) { return sendJson(res, 400, {
  error: e.message }); }
  const { situation_familiale,
situation_emploi, situation_admin } = body;
  const now = new Date().toISOString();
  db.prepare(`INSERT INTO profiles
(user_id, situation_familiale,
situation_emploi, situation_admin,
updated_at) VALUES (?, ?, ?, ?, ?)
  ON CONFLICT(user_id) DO UPDATE SET
situation_familiale=excluded.situation_fami
liale,
situation_emploi=excluded.situation_emploi,
situation_admin=excluded.situation_admin,
updated_at=excluded.updated_at`)
    .run(ctx.user.id, situation_familiale
|| null, situation_emploi || null,
situation_admin || null, now);
  return sendJson(res, 200, { ok: true });
}
function handleGetProfile(req, res, ctx) {
  if (!requireAuth(ctx, res)) return;
  const profile = db.prepare('SELECT * FROM
profiles WHERE user_id =
?').get(ctx.user.id) || null;
  return sendJson(res, 200, { user:
ctx.user, profile });
}
function handleListProcedures(req, res,
ctx, query) {
  const categorie = query.get('categorie');
  const rows = categorie
    ? db.prepare(`SELECT p.*, s.organisme,
s.url AS source_url, s.date_verification AS

```

```

source_date, s.statut AS source_statut FROM
procedures p LEFT JOIN sources s ON
s.id=p.source_id WHERE p.categorie=?
`).all(categorie)
    : db.prepare(`SELECT p.*, s.organisme,
s.url AS source_url, s.date_verification AS
source_date, s.statut AS source_statut FROM
procedures p LEFT JOIN sources s ON
s.id=p.source_id`).all();
    const withDocs = rows.map((r) => ({ ...r,
conditions: r.conditions_json ?
JSON.parse(r.conditions_json) : [],
documents: db.prepare('SELECT id, type,
obligatoire FROM documents WHERE
procedure_id=?').all(r.id) }));
    return sendJson(res, 200, { procedures:
withDocs });
}
function handleListSources(req, res) {
    return sendJson(res, 200, { sources:
db.prepare('SELECT * FROM sources').all()
});
}
async function handleCreateResult(req, res,
ctx) {
    if (!requireAuth(ctx, res)) return;
    let body;
    try { body = await readJsonBody(req); }
catch (e) { return sendJson(res, 400, {
error: e.message }); }
    const { objectif, situation, main_path,
autonomy_level, documents } = body;
    const id = 'result_' +
crypto.randomBytes(10).toString('hex');

```

```

    const now = new Date().toISOString();
    db.prepare(`INSERT INTO results (id,
user_id, objectif, situation_json,
main_path, autonomy_level, documents_json,
created_at, updated_at) VALUES
(?,?,?,?,?,?,?,?,?)`)
    .run(id, ctx.user.id, objectif || null,
JSON.stringify(situation || {}), main_path
|| null, autonomy_level || null,
JSON.stringify(documents || []), now, now);
    return sendJson(res, 201, { id });
}

function handleListResults(req, res, ctx) {
    if (!requireAuth(ctx, res)) return;
    const rows = db.prepare('SELECT * FROM
results WHERE user_id=? ORDER BY created_at
DESC').all(ctx.user.id);
    return sendJson(res, 200, { results:
rows.map(r => ({ ...r, situation:
JSON.parse(r.situation_json || '{}'),
documents: JSON.parse(r.documents_json ||
'[]') }))) });
}

function handleGetResult(req, res, ctx, id)
{
    if (!requireAuth(ctx, res)) return;
    const row = db.prepare('SELECT * FROM
results WHERE id=? AND user_id=?').get(id,
ctx.user.id);
    if (!row) return sendJson(res, 404, {
error: 'Parcours introuvable.' });
    return sendJson(res, 200, { ...row,
situation: JSON.parse(row.situation_json ||
'{}'), documents:

```

```

JSON.parse(row.documents_json || '[]') });
}
async function
handleUpdateResultDocuments(req, res, ctx,
id) {
  if (!requireAuth(ctx, res)) return;
  let body;
  try { body = await readJsonBody(req); }
  catch (e) { return sendJson(res, 400, {
error: e.message }); }
  const row = db.prepare('SELECT * FROM
results WHERE id=? AND user_id=?').get(id,
ctx.user.id);
  if (!row) return sendJson(res, 404, {
error: 'Parcours introuvable.' });
  db.prepare('UPDATE results SET
documents_json=?, updated_at=? WHERE
id=?').run(JSON.stringify(body.documents ||
[]), new Date().toISOString(), id);
  return sendJson(res, 200, { ok: true });
}
function handleDeleteResult(req, res, ctx,
id) {
  if (!requireAuth(ctx, res)) return;
  const info = db.prepare('DELETE FROM
results WHERE id=? AND user_id=?').run(id,
ctx.user.id);
  if (info.changes === 0) return
sendJson(res, 404, { error: 'Parcours
introuvable.' });
  return sendJson(res, 200, { ok: true });
}

//

```

```

=====
=====
// HANDLERS - admin
//
=====
=====
const VALID_STATUTS = ['a_jour',
'a_verifier', 'expiree'];
function handleAdminStats(req, res, ctx) {
  if (!requireAdmin(ctx, res)) return;
  return sendJson(res, 200, {
    users: db.prepare('SELECT COUNT(*) AS n
FROM users').get().n,
    results: db.prepare('SELECT COUNT(*) AS
n FROM results').get().n,
    sourcesByStatut: db.prepare('SELECT
statut, COUNT(*) AS n FROM sources GROUP BY
statut').all(),
  });
}
function handleAdminListSources(req, res,
ctx) {
  if (!requireAdmin(ctx, res)) return;
  const rows = db.prepare(`SELECT s.*,
(SELECT COUNT(*) FROM procedures p WHERE
p.source_id=s.id) AS procedures_count FROM
sources s ORDER BY s.organisme`).all();
  return sendJson(res, 200, { sources: rows
});
}
async function handleAdminUpdateSource(req,
res, ctx, id) {
  if (!requireAdmin(ctx, res)) return;
  let body;

```

```

    try { body = await readJsonBody(req); }
    catch (e) { return sendJson(res, 400, {
    error: e.message }); }

    const existing = db.prepare('SELECT id
    FROM sources WHERE id=?').get(id);
    if (!existing) return sendJson(res, 404,
    { error: 'Source introuvable.' });
    const { statut, date_verification, url }
    = body;
    if (statut &&
    !VALID_STATUTS.includes(statut)) return
    sendJson(res, 400, { error: `Statut
    invalide. Valeurs possibles :
    ${VALID_STATUTS.join(', ')}.` });
    const fields = []; const values = [];
    if (statut) { fields.push('statut=?');
    values.push(statut); }
    if (date_verification) {
    fields.push('date_verification=?');
    values.push(date_verification); }
    if (url) { fields.push('url=?');
    values.push(url); }
    if (fields.length === 0) return
    sendJson(res, 400, { error: 'Aucune
    modification fournie.' });
    values.push(id);
    db.prepare(`UPDATE sources SET
    ${fields.join(', ')} WHERE id=?
    `).run(...values);
    return sendJson(res, 200, { ok: true });
  }
  function handleAdminListUsers(req, res,
  ctx) {
    if (!requireAdmin(ctx, res)) return;

```

```

    return sendJson(res, 200, { users:
db.prepare('SELECT id, email, nom,
is_admin, created_at FROM users ORDER BY
created_at DESC').all() });
}

//
=====
=====
// ROUTAGE
//
=====
=====
async function routeApi(req, res, ctx,
pathname, query) {
    const method = req.method;
    if (method === 'POST' && pathname ===
'/api/auth/signup') return
handleSignup(req, res);
    if (method === 'POST' && pathname ===
'/api/auth/login') return handleLogin(req,
res);
    if (method === 'POST' && pathname ===
'/api/auth/logout') return
handleLogout(req, res, ctx);
    if (method === 'GET' && pathname ===
'/api/auth/me') return handleMe(req, res,
ctx);
    if (method === 'PUT' && pathname ===
'/api/auth/password') return
handleChangePassword(req, res, ctx);

    if (method === 'GET' && pathname ===
'/api/profile') return

```



```

handleGetProfile(req, res, ctx);
  if (method === 'PUT' && pathname ===
'/api/profile') return
handleUpdateProfile(req, res, ctx);

  if (method === 'GET' && pathname ===
'/api/procedures') return
handleListProcedures(req, res, ctx, query);
  if (method === 'GET' && pathname ===
'/api/sources') return
handleListSources(req, res);

  if (method === 'POST' && pathname ===
'/api/results') return
handleCreateResult(req, res, ctx);
  if (method === 'GET' && pathname ===
'/api/results') return
handleListResults(req, res, ctx);

  const resultMatch =
pathname.match(/^\/api\/results\/([a-zA-Z0-9_]+)$/);
  if (resultMatch && method === 'GET')
return handleGetResult(req, res, ctx,
resultMatch[1]);
  if (resultMatch && method === 'DELETE')
return handleDeleteResult(req, res, ctx,
resultMatch[1]);

  const docsMatch =
pathname.match(/^\/api\/results\/([a-zA-Z0-9_]+)\/documents$/);
  if (docsMatch && method === 'PUT') return
handleUpdateResultDocuments(req, res, ctx,

```

```

docsMatch[1]);

    if (method === 'GET' && pathname ===
'/api/admin/stats') return
handleAdminStats(req, res, ctx);
    if (method === 'GET' && pathname ===
'/api/admin/sources') return
handleAdminListSources(req, res, ctx);
    if (method === 'GET' && pathname ===
'/api/admin/users') return
handleAdminListUsers(req, res, ctx);
    const adminSourceMatch =
pathname.match(/^\/api\/admin\/sources\/([a
-zA-Z0-9_]+)$/);
    if (adminSourceMatch && method === 'PUT')
return handleAdminUpdateSource(req, res,
ctx, adminSourceMatch[1]);

    sendJson(res, 404, { error: 'Route API
inconnue.' });
}

//
=====
// FICHIERS STATIQUES + SÉCURITÉ + RATE
LIMIT
//
=====
const MIME_TYPES = { '.html': 'text/html;
charset=utf-8', '.js': 'text/javascript;
charset=utf-8', '.css': 'text/css;
charset=utf-8', '.json': 'application/json;

```

```

charset=utf-8', '.svg': 'image/svg+xml',
'.png': 'image/png', '.ico': 'image/x-icon'
});
function serveStatic(req, res, pathname) {
  let filePath = pathname === '/' ?
'/index.html' : pathname;
  filePath =
path.normalize(filePath).replace(/^(\\.\\.
[/\\])+/, '');
  const fullPath = path.join(PUBLIC_DIR,
filePath);
  if (!fullPath.startsWith(PUBLIC_DIR)) {
res.writeHead(403); res.end('Forbidden');
return; }
  fs.readFile(fullPath, (err, data) => {
    if (err) {
      fs.readFile(path.join(PUBLIC_DIR,
'index.html'), (err2, data2) => {
        if (err2) { res.writeHead(404);
res.end('Not found'); return; }
        res.writeHead(200, { 'Content-
Type': MIME_TYPES['.html'] });
        res.end(data2);
      });
      return;
    }
    const ext = path.extname(fullPath);
    res.writeHead(200, { 'Content-Type':
MIME_TYPES[ext] || 'application/octet-
stream' });
    res.end(data);
  });
}
function applySecurityHeaders(res) {

```

```

    res.setHeader('X-Content-Type-Options',
'nosniff');
    res.setHeader('X-Frame-Options', 'DENY');
    res.setHeader('Referrer-Policy', 'strict-
origin-when-cross-origin');
    if (IS_PROD) res.setHeader('Strict-
Transport-Security', 'max-age=15552000;
includeSubDomains');
}
const rateBuckets = new Map();
const RATE_LIMIT_WINDOW_MS = 60_000;
const RATE_LIMIT_MAX = 20;
function isRateLimited(ip) {
    const now = Date.now();
    const bucket = rateBuckets.get(ip);
    if (!bucket || bucket.resetAt < now) {
rateBuckets.set(ip, { count: 1, resetAt:
now + RATE_LIMIT_WINDOW_MS }); return
false; }
    bucket.count += 1;
    return bucket.count > RATE_LIMIT_MAX;
}
setInterval(() => { const now = Date.now();
for (const [ip, b] of rateBuckets) if
(b.resetAt < now) rateBuckets.delete(ip);
}, 5 * 60_000).unref();

function logRequest(req, res, startedAt,
ip) {
    console.log(JSON.stringify({ ts: new
Date().toISOString(), method: req.method,
path: req.url, status: res.statusCode,
durationMs: Date.now() - startedAt, ip }));
}

```

```
//
=====
=====
// SERVEUR
//
=====
=====
const server = http.createServer(async
(req, res) => {
  const startedAt = Date.now();
  applySecurityHeaders(res);
  const ip = (req.headers['x-forwarded-
for'] || req.socket.remoteAddress ||
'unknown').split(',')[0].trim();
  const url = new URL(req.url,
`http://${req.headers.host ||
'localhost'}`);
  const pathname = url.pathname;
  res.on('finish', () => logRequest(req,
res, startedAt, ip));

  if (pathname === '/api/health') {
    let dbOk = true;
    try { db.prepare('SELECT 1').get(); }
    catch (e) { dbOk = false; }
    return sendJson(res, dbOk ? 200 : 503,
{ status: dbOk ? 'ok' : 'degraded',
uptimeSeconds:
Math.round(process.uptime()), db: dbOk ?
'ok' : 'unreachable' });
  }

  if (pathname.startsWith('/api/')) {
```

```

        if (pathname.startsWith('/api/auth/')
        && isRateLimited(ip)) return sendJson(res,
        429, { error: 'Trop de tentatives,
        réessayez dans une minute.' });
        const cookies = parseCookies(req);
        const user =
        getUserBySessionToken(cookies.vlf_session);
        const ctx = { user, ip };
        try { await routeApi(req, res, ctx,
        pathname, url.searchParams); }
        catch (err) { console.error('Erreur
        API', pathname, err); if (!res.headersSent)
        sendJson(res, 500, { error: 'Erreur interne
        du serveur.' }); }
        return;
    }

    if (req.method === 'GET' || req.method
    === 'HEAD') serveStatic(req, res,
    pathname);
    else { res.writeHead(405);
    res.end('Method not allowed'); }
});

server.listen(PORT, () => {
    console.log(`VLF MVP en écoute sur le
    port ${PORT}`);
    console.log('Compte de démonstration :
    demo@vlf.fr / demo1234');
    console.log('Compte administrateur :
    admin@vlf.fr / admin1234');
});

let shuttingDown = false;

```

```
function gracefulShutdown(signal) {
  if (shuttingDown) return;
  shuttingDown = true;
  console.log(`\n${signal} reçu - arrêt
propre du serveur...`);
  server.close(() => { try { db.close(); }
catch (e) {} process.exit(0); });
  setTimeout(() => process.exit(1),
5000).unref();
}
process.on('SIGTERM', () =>
gracefulShutdown('SIGTERM'));
process.on('SIGINT', () =>
gracefulShutdown('SIGINT'));
```